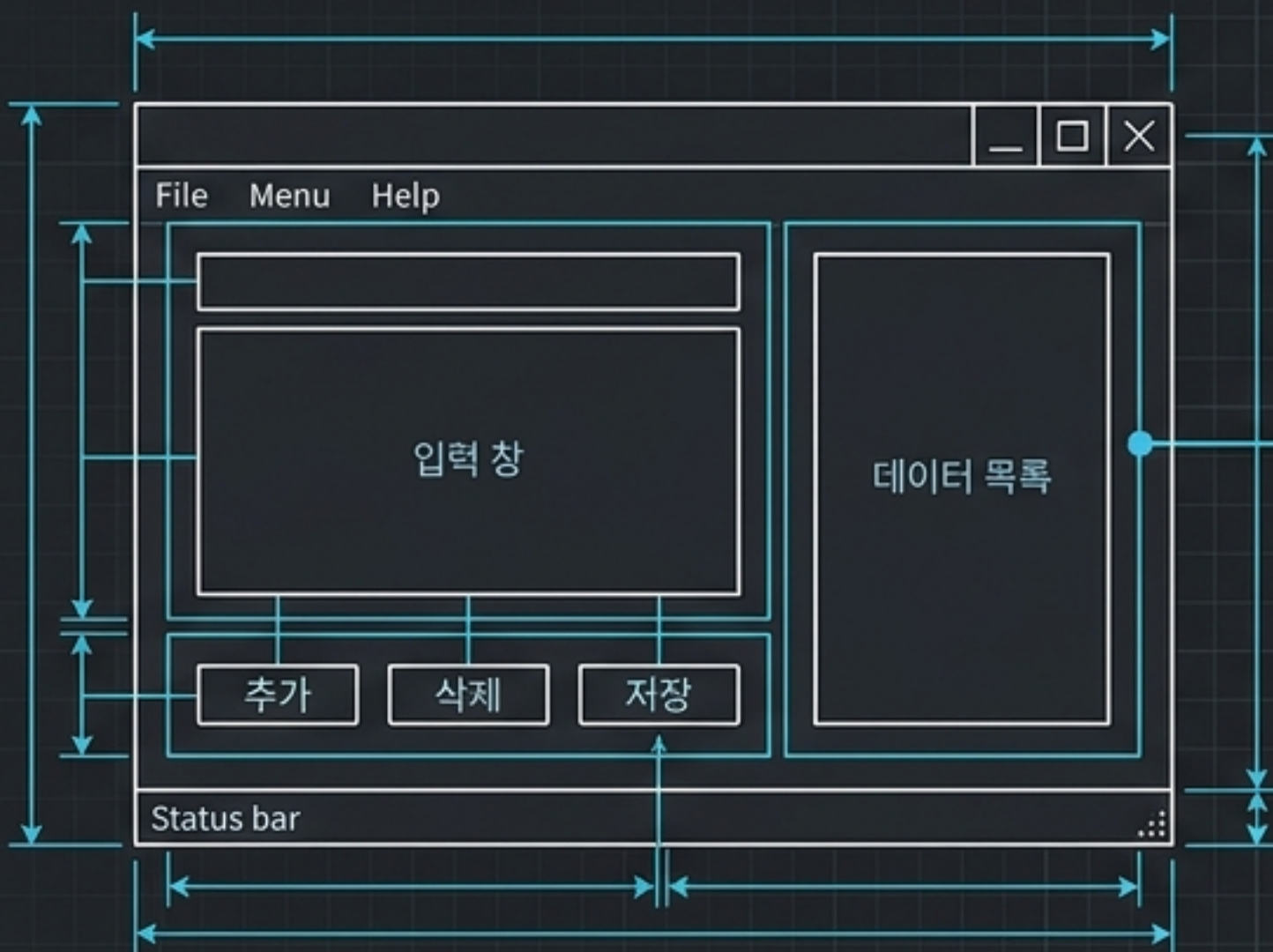


C# 윈도우 폼(Windows Forms) 마스터 클래스

GUI 프로그래밍의 첫걸음:
드래그 앤 드롭에서 동적 코드 생성까지



```
using System;
using System.Windows.Forms;

namespace WindowsFormsApp {
    public class MainForm : Form {

        private Button addButton;
        private TextBox inputBox;
        private ListBox dataList;

        public MainForm() {
            this.Text = "나의 첫 윈도우 폼";
            this.Size = new System.Drawing.Size(600, 450);

            inputBox = new TextBox();
            inputBox.Location = new System.Drawing.Point(10, 10);

            addButton = new Button();
            addButton.Text = "추가";
            addButton.Click += new EventHandler(AddButton_Click);

            dataList = new ListBox();
            // 동적 코드 생성 예시
            Controls.Add(inputBox);
            Controls.Add(addButton);
            Controls.Add(dataList);
        }

        private void AddButton_Click(object sender, EventArgs e) {
            // 데이터 처리 로직
            dataList.Items.Add(inputBox.Text);
        }
    }
}
```

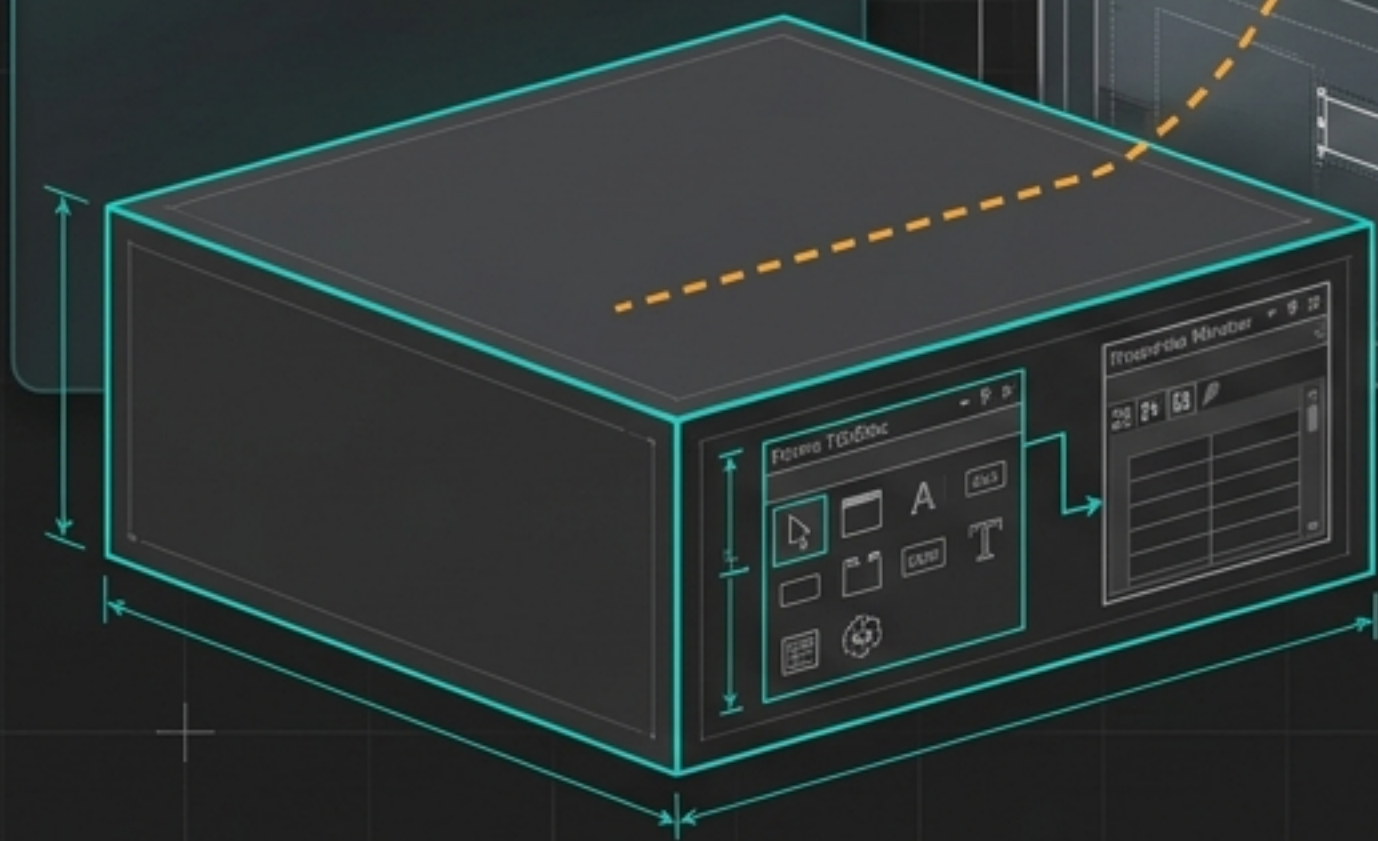
[.NET 8.0]

[Windows Forms]

윈도 폼을 지배하는 3가지 차원

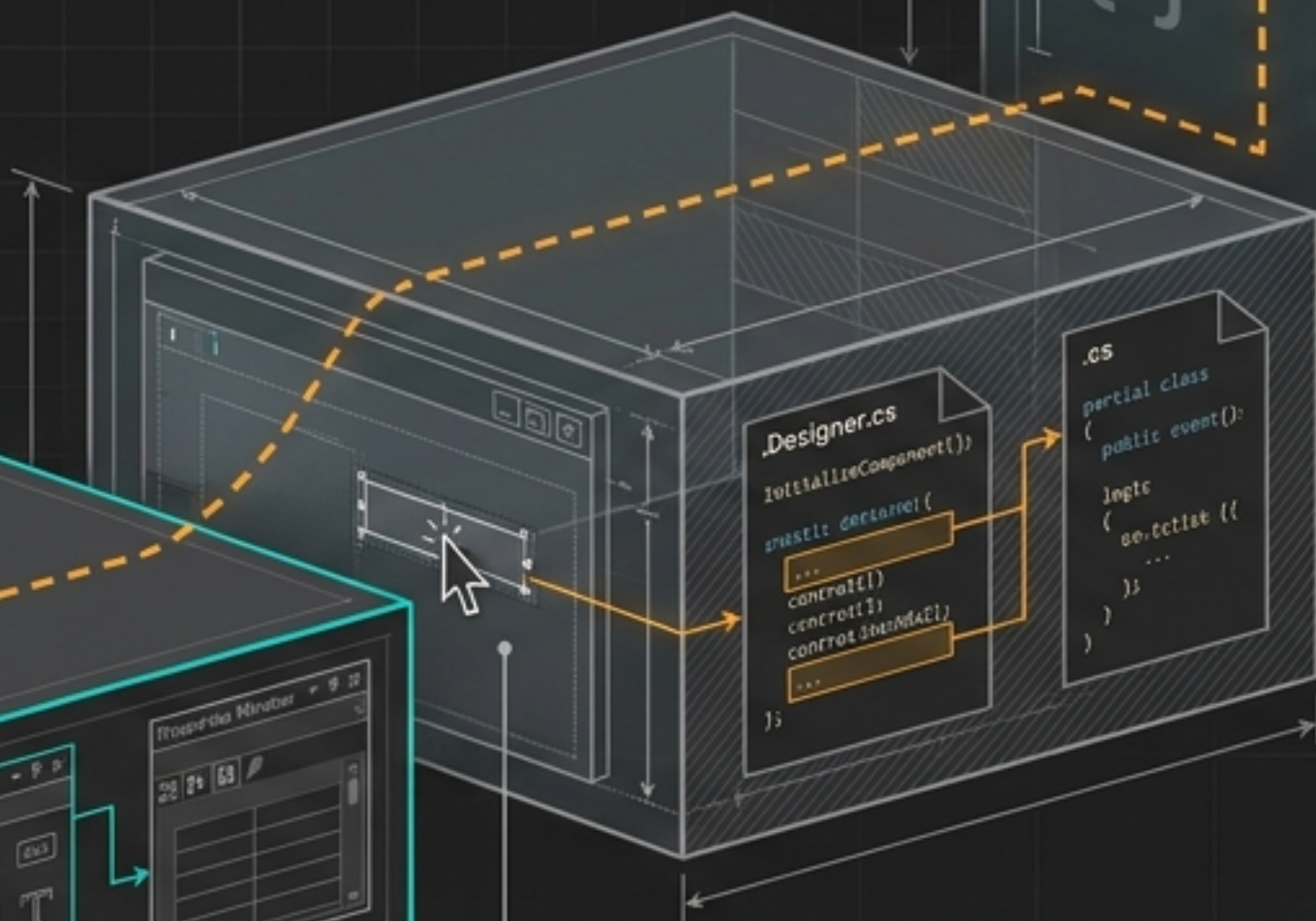
1. 표면 (Visual Design)

도구 상자와 속성 창을 활용한 직관적인 UI 설계



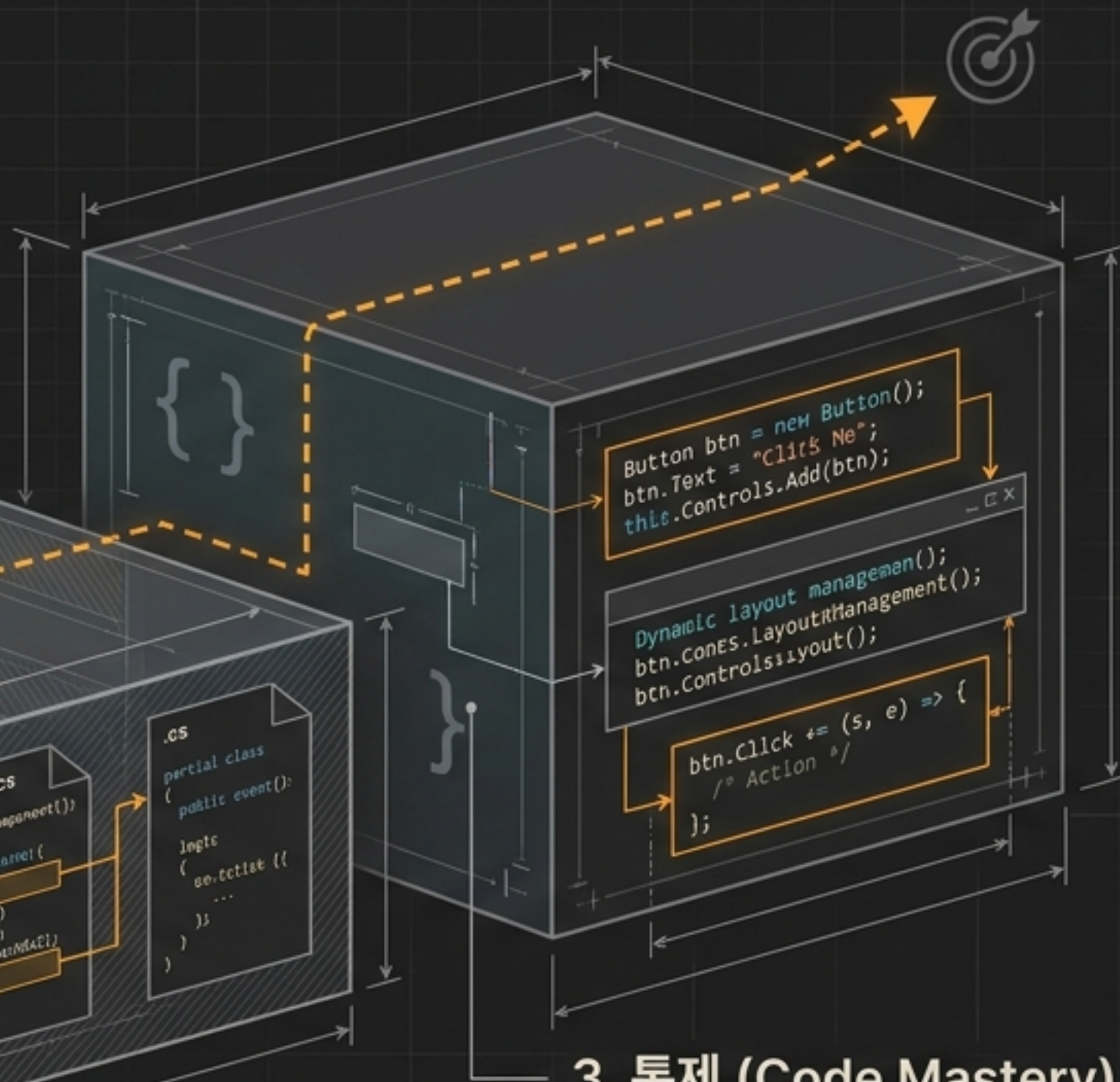
2. 이면 (Under the Hood)

마우스 클릭이 코드로 번역되는 디자이너 파일과 partial 클래스 해부



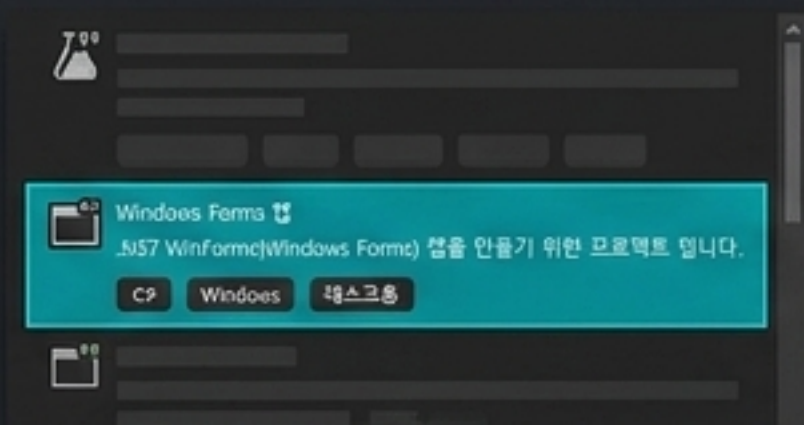
3. 통제 (Code Mastery)

마우스 없이 순수 코드로만 UI를 동적 생성하고 제어하기



프로젝트 생성: 새로운 캔버스 준비하기

1. 프로젝트 템플릿 선택



2. 직관적인 명명

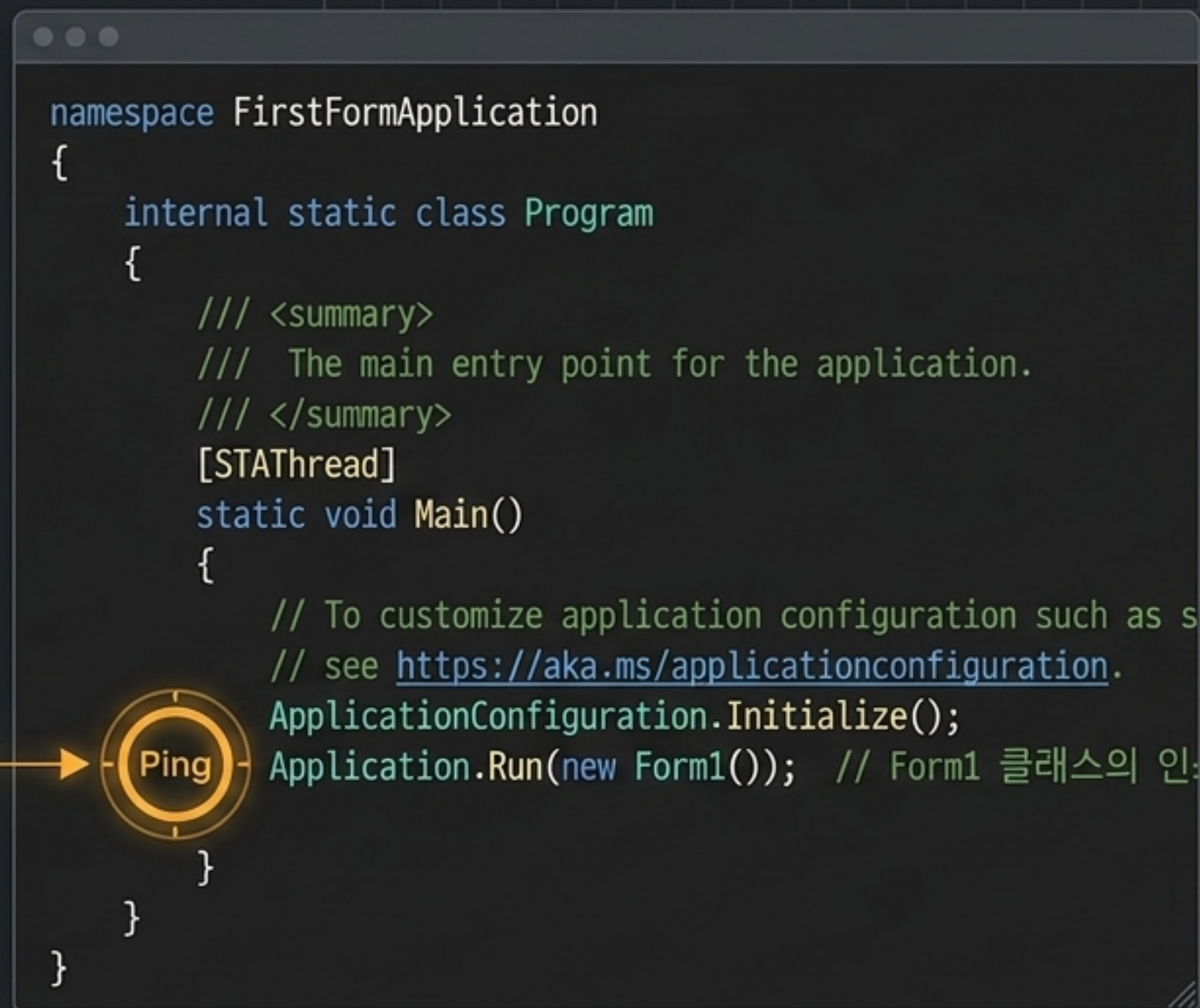
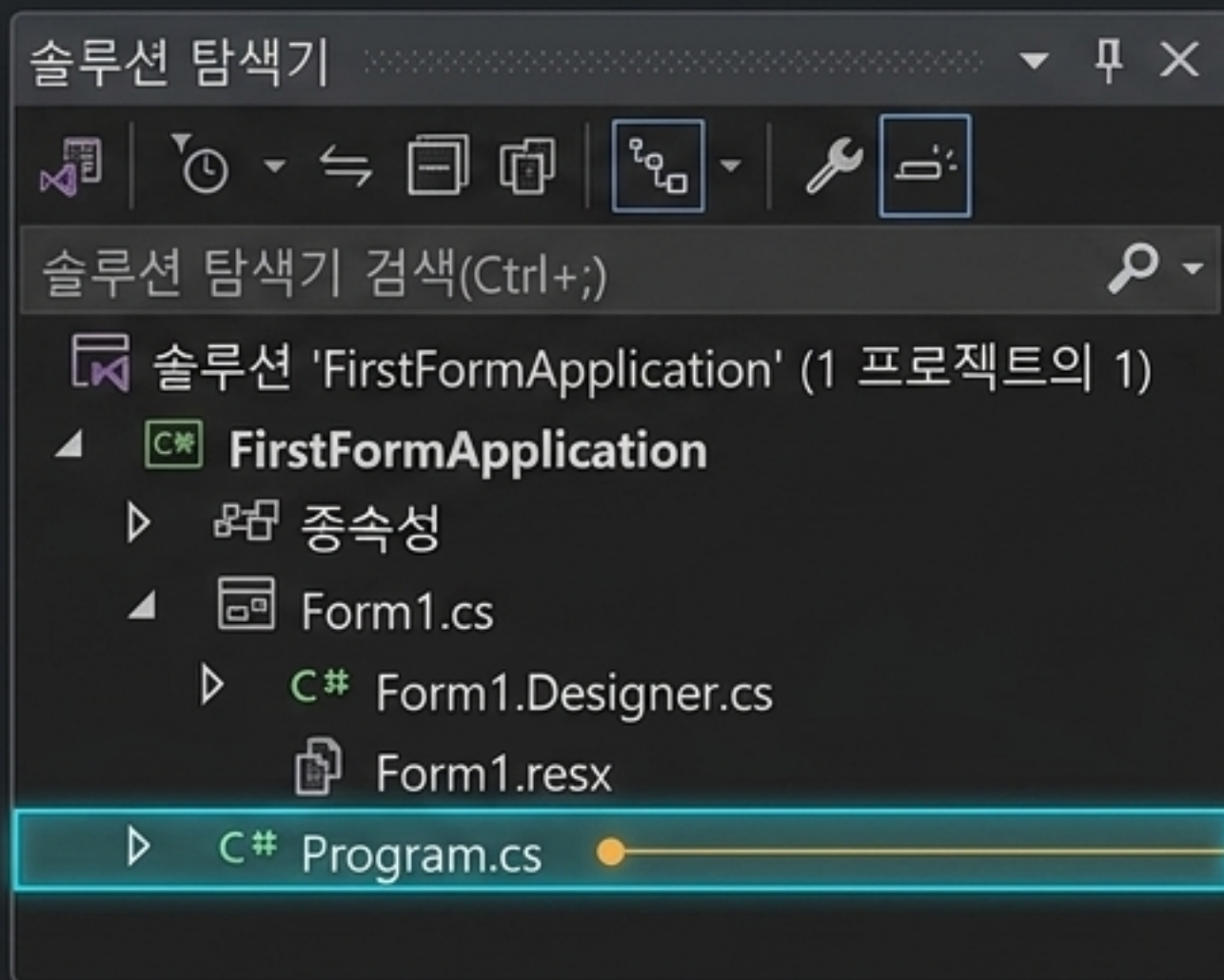
FirstFormApplication

3. 안정적인 프레임워크 선택

.NET 8.0 (장기 지원)

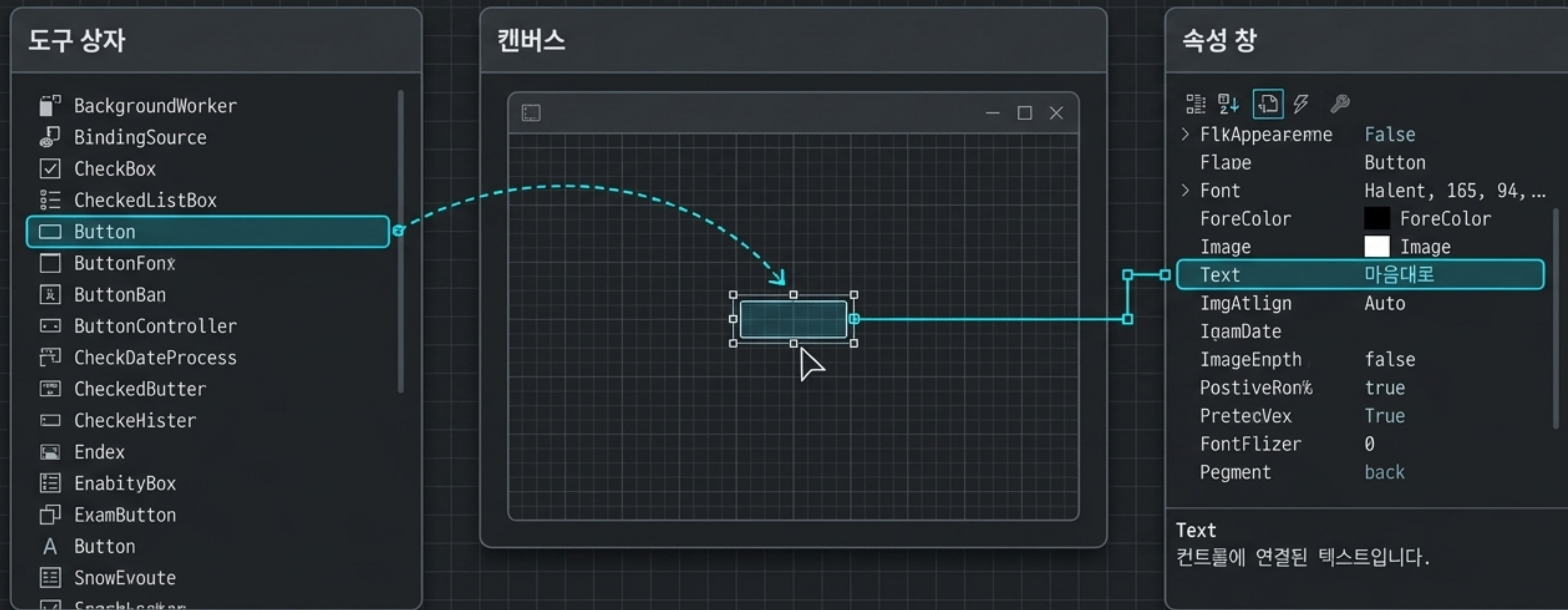
프로그램의 시작점, Program.cs

Main() 메서드에서 new Form1() 인스턴스를 생성하여 화면에 띄우는 구조적 진입점입니다.



직관적인 설계: 도구 상자와 속성 창

코딩 한 줄 없이 마우스만으로 객체를 캔버스에 배치하고 식별자(Name)와 표시 값(Text)을 부여합니다.



우리가 마우스로 한 일, 코드는 알고 있다

마음대로

디자인 화면에서 생성한 버튼은 **마법이 아닙니다.**
백그라운드에서 **실시간으로 완벽한 C# 코드로** 번역되고 있습니다.

무대 뒤의 비밀: Form1.Designer.cs

```
31 myButton = new Button();
32 SuspendLayout();
33 //
34 // myButton
35 //
36 myButton.Location = new Point(4, 5);
37 myButton.Name = "myButton";
38 myButton.Size = new Size(75, 23);
39 myButton.TabIndex = 0;
40 myButton.Text = "마음대로";
41 myButton.UseVisualStyleBackColor = true;
42 //
43 // Form1
44 //
45 AutoScaleDimensions = new.SizeF(10F, 25F);
46 AutoScaleMode = AutoScaleMode.Font;
47 ClientSize = new Size(800, 450);
48 Controls.Add(myButton);
49 Name = "Form1";
50 Text = "Form1";
51 Resizable = false;
52 }
53
54 #region Windows Form Designer generated code
55
56 private Button myButton;
```

● 객체 인스턴스화

● 화면에 부착



경고: do not modify - 이곳의 코드는 디자이너가 자동 관리하므로 직접 수정하지 마세요.

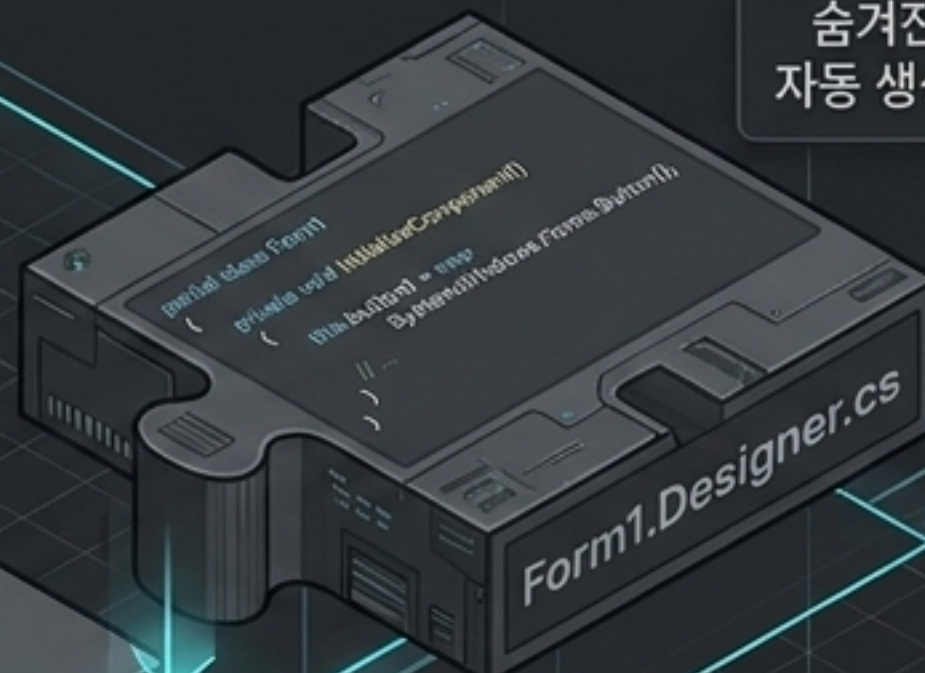
두 세계를 하나로 연결하는 마법: partial

partial 키워드는 클래스를 여러 파일에 분할 정의할 수 있게 해줍니다. 지저분한 UI 생성 코드는 숨기고 핵심 로직에만 집중하게 만드는 설계의 핵심입니다.

우리가 작성하는
깔끔한 사용자 로직



숨겨진 복잡한
자동 생성 UI 코드



컴파일 시 하나의 완전한 품으로 결합

코드에서 객체 속성 덮어쓰기

자동 생성된 객체(myButton)는 InitializeComponent() 호출 직후부터 우리가 마음대로 제어할 수 있습니다.
객체 지향의 편의성(IntelliSense)을 적극 활용하세요.

```
01 public partial class Form1 : Form
02 {
03     public Form1()
04     {
05         InitializeComponent();
06
07     myButton.Text = "코드에서 변경!";
08     }
09 }
```

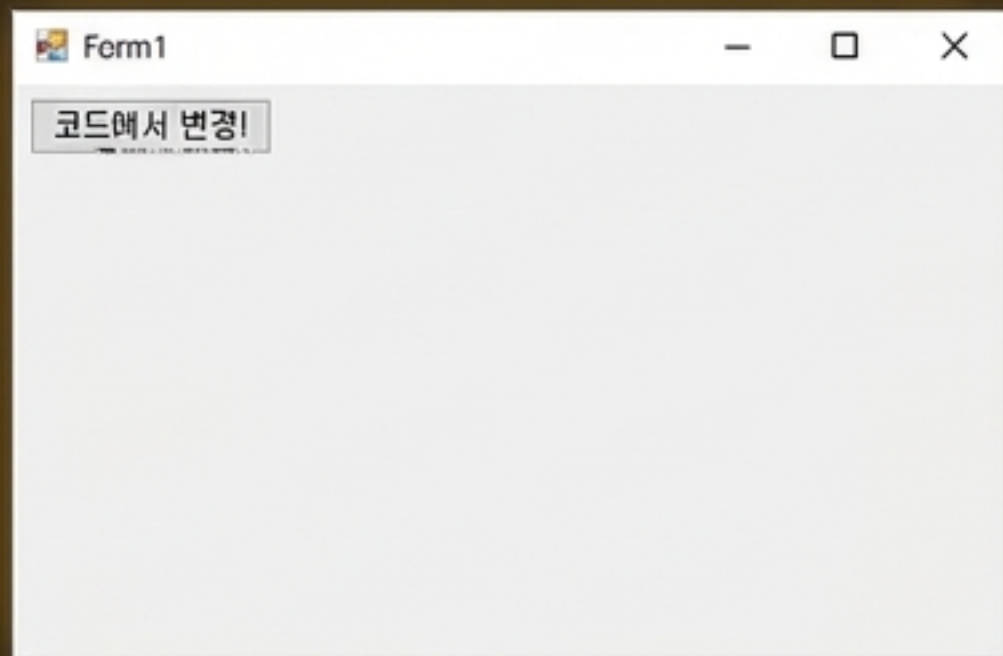
myButton.

- ★ Enabled
- ★ Anchor
- ★ Location
- ★ Text
- ★ DialogResult
- AccessibilityObject
- AccessibleDefaultActionDescription
- AccessibleDescription

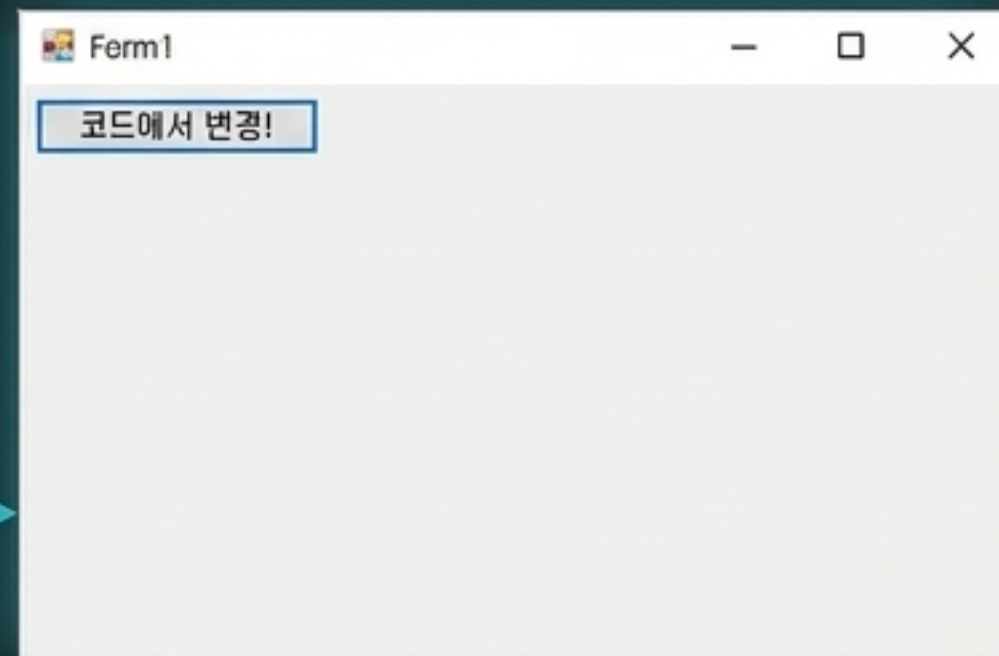
bool Control.Text { get; set; }
컨트롤의 텍스트를 가져오거나 설정합니다.
★ 이 컨텍스트를 기반으로 한 IntelliCode 제안

실무 주의: 디스플레이 배율과 UI 깨짐 현상

배율 불일치로 깨진 UI



정상 출력 UI



모니터 배율 100% 환경 기준, 버튼의 기본 높이는 23입니다. 다른 배율 환경에서는 출력이 잘리거나 깨질 수 있습니다.

해결책: 코드에서 명시적 크기 지정

```
myButton.Width = 100;  
myButton.Height = 23;
```

순수 코드로 창조하기

디자이너의 도움 없이 객체의 생명주기 3단계를 코드로 직접 구현합니다. 이 원리를 이해하면 진정한 GUI 통제력을 갖게 됩니다.

1

생성 (Instantiate)

```
Button button = new Button();
```



2

정의 (Define)

```
button.Location = new Point(x, y);  
button.Text = "...";
```



3

부착 (Attach)

```
Controls.Add(button);
```

코드가 디자인을 압도하는 순간: 자동화

반복문(for)을 통한 동적 생성 로직

```
01 public partial class Form1 : Form
02 {
03     public Form1()
04     {
05         InitializeComponent();
06         // 숫자를 적절하게 조절해서 사용해주세요.
07         int width = 100;
08         int height = 23;
09         int margin = 6;
10         myButton.Text = "코드에서 변경!";
11         myButton.Width = width;
12
13         for (int i = 0; i < 5; i++)
14         {
15             Button button = new Button();
16             Controls.Add(button);
17             button.Location = new Point(margin, (height + margin) * (i + 1) + margin);
18             button.Text = "동적 생성 " + i + "번째";
19             button.Width = width;
20             button.Height = height;
21         }
22     }
23 }
```

동적으로 렌더링된 결과물

Form1

동적 생성 0번째

동적 생성 1번째

동적 생성 2번째

동적 생성 3번째

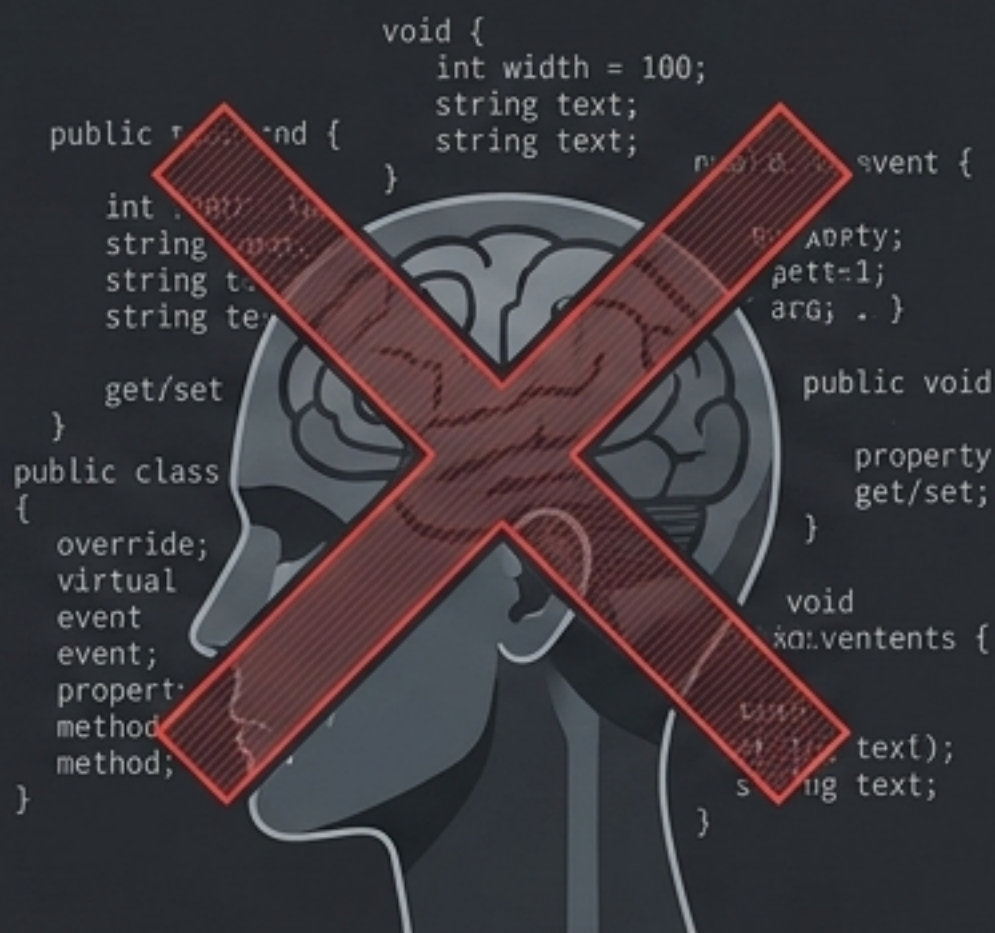
동적 생성 4번째

아키텍처 선택 매트릭스: 어떤 방식을 사용해야 할까?

	도구 상자 (디자이너) 방식	동적 코드 생성 방식
특징	✓ 빠르고 직관적임 (WYSIWYG)	✓ 무한한 유연성, 프로그래밍적 논리 제어
최적의 사용처	✓ 고정된 레이아웃, 정적인 폼, 빠른 프로토타이핑에 적합	✓ 데이터 수에 따라 변하는 가변적 UI, 복잡한 반복 요소 일괄 처리에 적합

상황에 따라 두 방식을 혼합(Hybrid)하여 사용하는 것이 가장 강력한 설계 방법입니다.

방대한 프레임워크를 대하는 자세



단순 암기

모든 속성을 외우려 하지 마세요.



실험과 빌드 (Build & Test)

요소를 넣고, 속성을 변경하는 '흐름'을 체득하세요.
직접 코드를 입력하고 부딪히며 결과를 확인하는 것이
가장 빠른 지름길입니다.

Key Takeaways (3줄 요약)

